

Motivation

Rust's crate ecosystem (`crates.io`) contains over **264,000 crates** with billions of downloads, however it is understudied compared to other ecosystems like npm and PyPI. Supply chain attacks are practical and effective, and become more popular year over year, a single compromised crate can propagate code to millions of downstream projects.

Key risks in the Rust supply chain:

- ✦ `build.rs` scripts allows arbitrary code execution at compile time (network, process spawning, etc.)
- ✦ Typosquatting and malicious crate uploads already observed in the wild in other ecosystems
- ✦ No existing framework integrates vulnerability scanning, secret detection, build script heuristics, typosquatting, and LLM-based analysis into a single pipeline

Objectives & Scope

Core Objective

Design, implement and evaluate a scalable security analysis platform that unifies six analysis modules and processes the entire crates.io ecosystem.

Scope

- ✦ Over 264,000 crates from the official crates.io database dump
- ✦ Source code analysis via shallow git clones of the latest commit
- ✦ 6 integrated analysis modules in parallel execution
- ✦ Redis-based distributed work queue with topological ordering
- ✦ Optimized PostgreSQL database
- ✦ Web dashboard
- ✦ Containerized deployment (Docker Compose, scalable worker count)

System Architecture

Data Pipeline

1. Parse **crates.io** CSV dump to extract crate metadata & dependency graph
2. Populate Redis queue with **leaf crates** (no dependencies first)
3. Workers claim crates via `BRPOPLPUSH`, clone source, run the analysis modules in parallel
4. Mark crate done, promote dependents via Lua scripts
5. Periodic cargo cache cleaning, results stored in PostgreSQL

Technology Stack

- ✦ **Core**: Rust (tokio, clap CLI, diesel ORM)
- ✦ **Queue**: Redis (lists, sets, Lua scripting for atomic operations)
- ✦ **DB**: PostgreSQL (7 tables, indexes)
- ✦ **Deploy**: Docker Compose, tmpfs ramdisk, multi-stage Dockerfiles
- ✦ **Web**: Axum + Tower-HTTP REST API, backend rendering, vanilla JS dashboard with Chart.js

Six Integrated Analysis Modules

1. Cargo audit Vulnerability Scanning

Cross-references the `Cargo.lock` file against the RustSec Advisory Database. Computes CVSS 3.1 scores. Detects unmaintained and yanked crates besides vulnerabilities.

2. Gitleaks Secret Detection

Scans source for 150+ secret patterns (API keys, tokens, private keys, credentials). Excludes test and example directories. Generates per-file findings with line numbers.

3. Build.rs Heuristic Analysis

Six rule-based checks on build scripts:

- ✦ Network calls (URLs, IPs, sockets)
- ✦ Process spawning (`Command`, `std::process`)
- ✦ Raw IP addresses and suspicious TLDs (`.tk`, `.xyz`, etc.)
- ✦ High-entropy strings (possible obfuscation)

4. LLM-Based Malicious Code Scoring

Locally-hosted LM Studio model scores crate source for malicious intent. Treated as an optional module

5. Typosquatting Detection

Five string-similarity algorithms combined with weighted scoring: Levenshtein, Damerau-Levenshtein, Jaro-Winkler, keyboard proximity, prefix similarity. Compares every crate name against all others.

6. Transitive Risk Propagation

Dependency graph analysis that propagates vulnerability and risk scores **transitively** through the entire dependency tree.

Performance Characteristics

Per-crate analysis time (226,546 crates)	
Metric	Value
Median	1.46 s
Mean	15.1 s
P95	86.8 s
Max	52.1 min
Bottleneck	<code>cargo-audit</code> (74% of mean time)

Horizontal scaling with topological ordering ensures dependencies are always processed before their dependents. Cycle detection via DFS at seeding time.

Key Findings

Dataset

226,546 crates scanned successfully (85.6% of 264,750 total).
13% failed due to missing `Cargo.lock` or inaccessible repositories.

Vulnerabilities

- ✦ **32.1%** of scanned crates have at least one known vulnerability (72,773 crates)
- ✦ Most frequent advisory: RUSTSEC-2023-0071 (12,487 occurrences)
- ✦ **82.6%** informational severity; only 11% High/Critical
- ✦ Crate age < 6 months: **30.9% vuln rate** (higher than average)
- ✦ High-download-velocity crates: only **25.2% vuln rate**

Secrets & Build Scripts

- ✦ 882,955 secrets found across 18,571 crates
- ✦ 95.8% generic-api-key matches (almost all are false positives)
- ✦ 561 crates leak GitHub PATs
- ✦ 3,194 crates (1.41%) spawn processes in `build.rs`
- ✦ 1,005 crates (0.44%) make network calls in `build.rs`

Critical Insights

Crates shipping binaries have a **67.7% vulnerability rate**, more than double the ecosystem average (32.1%). **38,495** name-similarity pairs detected; **891 pairs** with scores ≥ 95 . However this is mostly due to crates.io crate naming patterns.

Substrate/Polkadot: #1 Risk Source

The largest single source of transitive vulnerability risk. `sp-runtime`: 567 dependents, 41 vulnerabilities. The top 100 most-vulnerable crates contain a large fraction of all ecosystem vulnerabilities. Th dependency network counts over **1.7 million** relationships.

Conclusions & Future Work

This thesis provides the **first large-scale, multi-modal security characterization of crates.io**. The framework is open-source and designed for potential Rust Foundation adoption, however with many required changes, and sufficiently powerful hardware.

Future Directions

- ✦ Version-specific analysis (track vulnerabilities per release)
- ✦ Real-time monitoring of new crate publications
- ✦ Sandboxed `build.rs` execution for dynamic analysis
- ✦ Repository caching layer to reduce git clone overhead
- ✦ Storing everything possible in RAM, more caching

References

- [1] Ladisa et al., *SoK: Taxonomy of Attacks on Open-Source Software Supply Chains*, IEEE S&P 2023.
- [2] Ohm et al., *Backstabber's Knife Collection*, IEEE S&P 2020.
- [3] Zimmermann et al., *Small World with High Risks*, USENIX Security 2019.
- [4] RustSec Advisory Database, <https://rustsec.org/>.